

FictiPay Churn Prediction — Model Performance Report

Cybernauts 2026 Datathon Submission

Pipeline v4 — 5-fold CV, LightGBM + CatBoost ensemble

1. Large-Scale Data Handling

The competition dataset comprises 200M+ transaction rows and roughly 360M day-end balance rows (~1.96 GB on disk), against an account universe of ~850,000 customer accounts. Loading either table fully into pandas would not fit in memory on the dual-T4 Kaggle environment, so all ingestion and row-level filtering is performed with **Dask**.

Framework. `dask.dataframe.read_parquet` is used with glob patterns (`trx_*.parquet`, `balance_*.parquet`) to read the partitioned parquet files lazily. Only the small reference tables — KYC (~1M rows), `train_labels.csv`, and `test.csv` — are loaded directly with pandas.

Partitioning and temporal filtering. Immediately after loading, both the transaction and balance Dask frames are filtered to `TRX_DATETIME < CUTOFF / DATE < CUTOFF` where `CUTOFF = 2024-04-01`. Because this filter is applied before any `.compute()` call, Dask can push the predicate down to the partition level — partitions entirely outside the observation window are skipped during evaluation, rather than being materialized and then discarded.

Sampling / aggregation strategy. No row-level sample of the raw data is ever materialized to pandas. Instead, every feature is produced via a `groupby(...).agg(...).compute()` (or `.size() / .nunique() / .min()`) call, which collapses the data to **one row per account** before leaving Dask. This means peak memory for any single aggregation step is bounded by the size of the *output* (~850K rows × a handful of columns), not by the size of the input (200M+ / 360M rows).

Single-pass consolidation. The weekly-binned activity block (Section 2c of the pipeline) is the clearest example of this strategy: rather than running three separate windowed aggregations for 7-day, 30-day, and 90-day counts/sums (three full passes over the filtered transaction frame), the pipeline bins `days_ago` into 13 weekly buckets (`week_bin = days_ago // 7`) and performs **one** `groupby(["SRC_ACCOUNT", "week_bin"]).agg(["count", "sum"]).compute()`. The 7d/30d/90d aggregates, the activity decay slope, and `weeks_since_active` are all derived from this single materialized 13-column-wide table in pandas afterward — no additional Dask compute is required.

The two pandas-side operations that do require a full per-row balance history (`bal_pd = bal_ddf[...].compute()`, used for slope calculations and the trailing-streak features) are unavoidable because `scipy.stats.linregress` and the run-length encoding both need the ordered daily series per account — but even here, the trailing-streak computation is

fully **vectorized** via `cumcount/cummax/transform`, avoiding a per-group Python-level `.apply` over ~850K groups.

2. Feature Engineering

See **features.md** for the complete feature catalogue (137 features across 18 categories, with rationale for each group). Summary:

Category	# Features
Transaction window aggregates (7d/30d/90d)	15
Fine-grained recent activity (1d/2d/3d/5d)	4
Overall recency	1
Transaction-type mix (90d)	10
Network breadth	2
P2P receipt activity	1
Per-type, per-direction recency	10
Weekly-binned activity & trend	34
Bill payment regularity	5
Balance level & distribution	8
Balance trend (full + 14-day)	2
Balance drain timing	1
Zero-balance activity	1
Recent vs. early balance	2
Trailing dormancy streaks	2
KYC / demographic	3
Interaction / ratio features	16
Missing-indicator flags	19
Total	137

3. Feature Quality & Pareto

Final matrix. 595,000 training rows × 137 columns; 255,000 test rows × 137 columns. Churn rate: **12.678%** (75,435 / 595,000), matching the README's stated overall split of 519,565 / 75,435.

Missing-value handling (two-tier strategy).

- **99 count-like columns** (counts, sums, recencies, binary flags) use a -999 sentinel. For these features, “missing” genuinely means “no recorded activity of this type” — a meaningful state that tree-based models can learn a dedicated split point for, rather than a value that should be smoothed away.
- **19 ratio/slope/momentum-type columns** (identified by name pattern: `ratio`, `slope`, `momentum`, `_cv`, `shift`, `_vs_`) are **median imputed** (median computed on the training set only, applied to both train and test), each paired with an explicit `_isnan` indicator column.

Infinity handling. Several ratio features divide by quantities that can be zero (e.g. `trx_count_90d` in `channel_shift_ratio`). This run includes the fix `train_feat.replace([np.inf, -np.inf], np.nan, inplace=True)` (and the same for `test_feat`) applied immediately at the start of the matrix-building step, **before** the `RATIO_SLOPE_COLS` median imputation — so any `inf/-inf` produced by a zero-denominator division is converted to `NaN` and handled by the same median-impute + `_isnan`-flag path as genuine missing values, rather than entering the model as an out-of-range numeric value.

Distributions. The dominant pattern across the count-like feature group is a large spike at -999 (accounts with zero activity in that specific window/channel) alongside a right-skewed positive distribution for active accounts — visible in `trx_count_30d`, `trx_count_7d`, and the per-channel recency columns. The balance-trend features (`bal_slope_14d`, `activity_decay_slope`) are approximately centered near zero with heavy tails in both directions, reflecting the mix of growing, shrinking, and stable accounts. The 13 weekly activity bins (`trx_count_w0..w12`) show a gradual decline in non-zero mass for higher week indices, consistent with the observation window covering a fixed historical period rather than a rolling one.

(Recommended figures to attach here, generated on Kaggle from `train_feat` before the -999/median fill: histograms of `trx_count_30d`, `bal_slope_14d`, and `weeks_since_active`, plus a bar chart of the 99/19/1 missing-value-strategy split above.)

Sparsity. 99 of 137 columns (72%) are count-like and -999-sentineled; a meaningful fraction of accounts (the $\sim 1,254 / 595,000 = 0.2\%$ with no transaction records at all, identified during diagnostics) are entirely -999 across the transaction-derived feature groups, relying on KYC and balance features for signal. This subgroup has an empirical churn rate of **99.5%**, the strongest single segment-level signal found during diagnostics.

4. Class Imbalance & Sampling

Quantification. The training set has a churn rate of 12.678% (75,435 positive / 595,000 total), consistent with the README’s stated 519,565 / 75,435 split for the full population.

Method chosen: class weighting, not resampling. Both LightGBM and CatBoost are trained with `scale_pos_weight = 519_565 / 75_435  $\approx$  6.888`, which up-weights the loss

contribution of the minority (churn) class during training, rather than duplicating minority rows (oversampling/SMOTE) or discarding majority rows (undersampling).

Justification. The competition’s evaluation metric is **AUC-ROC** on a *float* CHURN_PROB $\in [0, 1]$, with Average Precision, Precision@10%, and Recall@10% also reported. This has two implications that favor weighting over resampling:

1. **Probability calibration.** Resampling methods change the class distribution the model is trained on, which shifts the model’s output probability scale away from the true population base rate. Class weighting changes the *loss gradient*, not the data distribution itself, so the resulting probabilities remain anchored to the true 12.678% base rate — important when the output is consumed as a calibrated probability rather than just a ranking.
2. **No information loss or duplication.** With 200M+ underlying transaction rows already collapsed into ~850K account-level feature rows, oversampling the ~75K minority accounts would create exact duplicate feature vectors (since features are account-level aggregates, not per-transaction), which can encourage overfitting to those specific accounts’ feature values. Undersampling would discard a large fraction of the majority class’s information for free.

scale_pos_weight is the standard, low-risk approach for gradient-boosted trees under moderate imbalance (~7:1 here), and both LightGBM’s and CatBoost’s implementations apply it directly in the loss function without altering the training set.

5. Model Selection & Training (AUTO-scored)

Models. An ensemble of **LightGBM** (gbdt, leaf-wise growth, histogram-binned splits) and **CatBoost** (symmetric-tree, ordered boosting) is trained, on the rationale that the two algorithms’ different tree-growth strategies and split-finding methods produce complementary errors, which a simple weighted blend can average out.

Cross-validation design. StratifiedKFold(n_splits=5, random_state=42). Each fold uses **out-of-fold REGION target encoding**: within each fold, REGION is mapped to that fold’s training-set churn rate per region (global mean as fallback for unseen categories), and this encoding is applied to that fold’s validation set and the test set — ensuring no validation-fold label information leaks into the encoding.

Hyperparameters (both models).

Parameter	LightGBM	CatBoost
learning_rate	0.02	0.03
n_estimators / iterations	6000	4000
num_leaves / depth	255	8
min_child_samples	30	—

Parameter	LightGBM	CatBoost
reg_alpha / l2_leaf_reg	0.1	3.0
reg_lambda	1.0	—
feature_fraction / bagging_fraction	0.8 / 0.8 (freq=5)	—
scale_pos_weight	6.888	6.888
early_stopping_rounds	200	200

Per-fold results.

Fold	LGB AUC	LGB best_iter	CatBoost AUC	CB best_iter
1	0.98424	244	0.98423	478
2	0.98435	183	0.98437	237
3	0.98502	215	0.98496	385
4	0.98460	199	0.98464	601
5	0.98456	255	0.98459	355

Metric	Value
LightGBM OOF AUC	0.98444 (± 0.00027)
CatBoost OOF AUC	0.98454 (± 0.00025)

Blend search. A weighted blend $w \cdot \text{oof_lgb} + (1-w) \cdot \text{oof_cb}$ is searched over $w \in [0, 1]$ in steps of 0.05. Best result:

Metric	Value
Best blend weight (LightGBM)	0.45
Blended OOF AUC	0.98460
Blended OOF Average Precision	0.91982
Precision @ Top 10%	0.90975
Recall @ Top 10%	0.71757

Interpretation. The blend (slightly favoring CatBoost at 55% weight) outperforms either single model's OOF AUC (0.98444 LGB / 0.98454 CB $\rightarrow$ 0.98460 blended), confirming the two models make partially-independent errors. At the operating point of “flag the top 10% of accounts by predicted churn probability,” the model correctly flags **71.8% of all churners** (Recall@10%) while **91.0% of flagged accounts genuinely churn** (Precision@10%) — a high-confidence, low-noise targeting list suitable for retention campaign use (see `presentation.pptx` for the business framing).

Final submission. `predictions.csv` — 255,000 rows, predicted mean churn probability 0.1884, predicted-positive rate (>0.5) 20.252%.

Note: the executed notebook writes this file as `predictions_new.csv` (`OUTPUT_PATH` in the `CONFIG` cell). The annotated notebook `.ipynb` provided with this submission has `OUTPUT_PATH` updated to `"predictions.csv"` to match the submission requirements — rename the file accordingly if submitting the output of the original run as-is.

6. Hyperparameter Tuning

Configuration evaluated in this report. `learning_rate=0.02`, `num_leaves=255`, `n_estimators=6000`, `min_child_samples=30` (LightGBM); `learning_rate=0.03`, `depth=8`, `iterations=4000` (CatBoost); `StratifiedKFold(n_splits=5, random_state=42)`; `inf` → NaN fix applied before ratio-column imputation.

Tuning history. An earlier, lower-capacity configuration — `learning_rate=0.05`, `num_leaves=127`, `n_estimators=3000`, `min_child_samples=50`, same 5-fold/1-seed CV, single LightGBM model — converged in 78–113 boosting rounds with an OOF AUC of **0.98450** (public leaderboard 0.98480).

The configuration in this report roughly doubled model capacity and halved the learning rate, which correspondingly increased the number of boosting rounds used before early stopping to **183–255** for LightGBM (237–601 for CatBoost) — i.e. the lower learning rate is being used as intended (more, smaller steps) rather than triggering premature early-stopping.

Configuration	Best iterations (LGB)	Blended/Single OOF AUC
<code>lr=0.05</code> , <code>leaves=127</code> , <code>n_est=3000</code> (LGB only)	78–113	0.98450
<code>lr=0.02</code> , <code>leaves=255</code> , <code>n_est=6000</code> + CatBoost blend (this run)	183–255	0.98460

A separate experiment additionally repeated this same configuration with `StratifiedKFold(n_splits=10)` across 3 seeds (42, 123, 2024), averaging 30 fold-fits per model — that variant produced a blended OOF AUC of 0.98468, a further +0.00008 over the 5-fold/1-seed result reported here, at roughly 6× the training cost (60 vs. 10 model fits). Given the marginal gain relative to the added compute, the 5-fold/1-seed configuration in this report was selected as the final submission.

Search method. Hyperparameters were adjusted manually based on observed `best_iteration_` values from the previous configuration (78–113 rounds suggested the model was converging too quickly relative to its capacity under the original learning rate) — this is a manual, iteration-guided search rather than an automated sweep (e.g. Optuna/grid search), with each configuration's OOF AUC and per-fold breakdown tracked in the printed training log (reproduced in Section 5 above) before being adopted.

Tracking. All fold-level AUCs, best iterations, and the blend-weight search are printed during training and preserved in the executed notebook's cell outputs (`notebook.ipynb`), providing a full audit trail from the per-fold metric through to the final blended submission.

Remaining headroom. Across three independent feature-engineering and tuning iterations (0.98428 → 0.98480 → ~0.9846x public; 0.98450 → 0.98461 → 0.98460 OOF), gains have consistently been in the 0.0001–0.0006 range, suggesting this feature representation is near a practical ceiling for this model family. The error analysis in the explainability folder identifies the remaining hard cases — accounts that are *active with rising balances* yet still churn, and accounts that go *quiet with falling balances* yet do not churn — as the primary candidates for future feature work (e.g. last-transaction anomaly detection).